DEEP LEARNING

Neural networks, optimisation, architectures, and evaluation

1. What is deep learning?

Deep learning is a subfield of machine learning that uses neural networks with multiple layers to learn representations from data. Rather than relying entirely on manually designed features, a network can learn useful intermediate features during training.

Applications include image recognition, speech recognition, language modelling, recommendation, forecasting, and generative AI. Deep learning can require substantial data and compute, although smaller networks can work well for focused problems.

2. Artificial neurons and network layers

An artificial neuron receives numerical inputs, multiplies them by weights, adds a bias, and applies an activation function. A simplified computation is z = w■x■ + w■x■ + ... + b, followed by a = f(z). Here, x represents inputs, w represents learned weights, b is a bias, and f is an activation function.

Component	Role	Examples
Weights	Control the influence of inputs	Learned numerical parameters
Bias	Shifts the activation value	One bias per neuron
Activation	Adds non-linearity	ReLU, sigmoid, tanh
Layer	Groups neurons in a stage	Input, hidden, output
Loss function	Measures prediction error	Cross-entropy, MSE

3. Activation functions

Function	Behaviour	Notes
ReLU	max(0, x)	Common in hidden layers
Sigmoid	Maps values to (0, 1)	Often used for binary output probabilities
Tanh	Maps values to (-1, 1)	Zero-centred; may saturate
Softmax	Outputs values summing to 1	Multi-class probability output
GELU	Smooth non-linear activation	Common in transformer networks

4. How neural networks learn

Training repeatedly performs a forward pass, calculates a loss, computes gradients using backpropagation, and updates model parameters with an optimiser. This repeats across batches and epochs.

• Forward propagation computes predictions from current parameters.

• A loss function quantifies prediction error.

• Backpropagation applies the chain rule to calculate gradients.

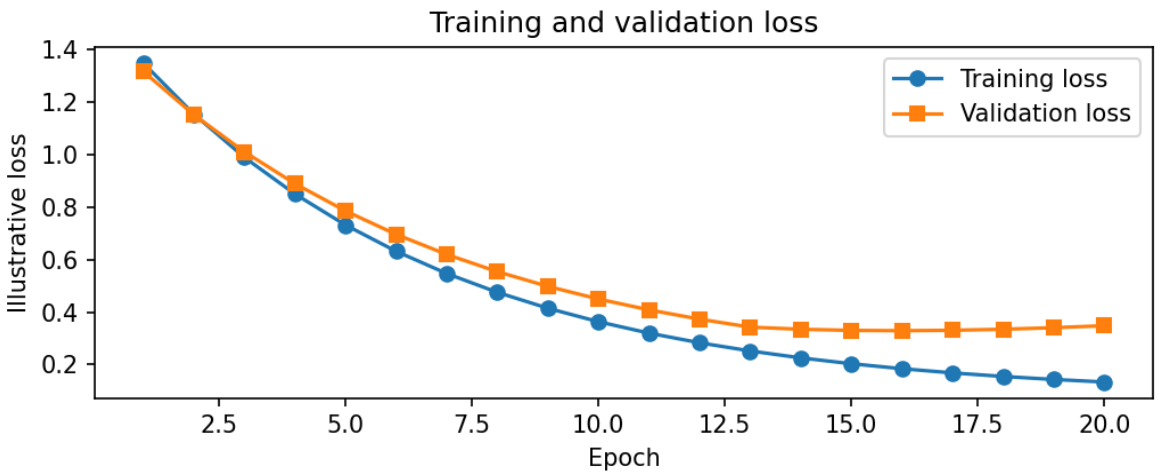
• An optimiser updates parameters, for example using SGD or Adam.

• A batch is a subset of examples processed before an update; an epoch is one pass through the training set.

5. Gradient descent and learning rate

Gradient descent adjusts parameters in a direction that reduces the loss. In a basic update, the parameter is reduced by the learning rate multiplied by its gradient. A learning rate that is too high can cause unstable updates; one that is too low can make training slow.

6. Training and validation curves



Illustrative curves only. If training loss decreases while validation loss begins to increase, the model may be overfitting. Early stopping can use validation performance to select a suitable training point.

7. Common architectures

Architecture	Core idea	Applications
MLP / feedforward	Fully connected layers	Tabular prediction
CNN	Convolutions learn local spatial patterns	Images and some audio tasks
RNN	Recurrent state processes sequences	Sequential data
LSTM / GRU	Gated recurrent sequence models	Speech and time series
Transformer	Attention models relationships between elements	LLMs, translation, vision
Autoencoder	Learns compressed representations and reconstruction	Denoising, anomaly detection
GAN	Generator and discriminator train adversarially	Synthetic data generation
Diffusion	Learns to reverse a gradual noising process	Image and audio generation

8. CNNs and transformers

A convolutional neural network applies learnable filters across an input. Early layers can detect simple patterns, while deeper layers can combine them into more complex features. Pooling or strided operations can reduce spatial dimensions.

A transformer uses attention to combine information from different positions. Self-attention calculates how strongly each token should use information from other tokens. Positional information is added because attention alone does not inherently encode sequence order.

9. Overfitting and regularisation

Overfitting occurs when a model fits training examples too closely and performs poorly on unseen data. Regularisation methods aim to improve generalisation.

Technique	Description
Dropout	Randomly disables some activations during training.
Weight decay	Penalises large parameter values.
Data augmentation	Creates plausible variations of training examples.
Early stopping	Stops training when validation performance stops improving.
Batch normalisation	Normalises intermediate activations using batch statistics during training.

10. Evaluation and deployment

Classification tasks may use accuracy, precision, recall, F1-score, and ROC-AUC. Regression tasks may use MAE, MSE, or RMSE. Generative tasks often require automatic metrics plus human evaluation and task-specific tests.

Production systems also need to meet constraints involving latency, memory, throughput, cost, privacy, and reliability. Monitor changes in input data and performance after deployment.

11. Key terms

Tensor: A multidimensional numerical array. **Parameter:** A value learned during training. **Hyperparameter:** A setting selected outside parameter learning. **Gradient:** A derivative indicating how a quantity changes with parameters. **Backpropagation:** Gradient computation through a network. **Transfer learning:** Reusing a model trained on one task or dataset for another.

Summary

Deep learning trains layered neural networks to learn representations. Its foundations include neurons, activations, loss functions, backpropagation, optimisation, regularisation, and evaluation. Architecture choice should reflect the data and application.